DSAA5022 Course Project

以太坊交易行为分析系统

Part A 技术报告

2026 年 5 月

摘要

这次项目做的是以太坊交易数据的欺诈检测。我们拿到了一个真实数据集，里面有 9841 个以太坊地址，其中 22% 已经被标记为欺诈地址。说实话，这个数据的比例比我想象中高不少——一般的欺诈检测数据集里欺诈样本通常不到 1%，这个能有 22%，算是比较” 友好” 了。

我们在这份报告里详细记录了从数据清洗到模型训练的全过程。包括怎么处理那些带空格的列名、怎么设计新特征、以及怎么用随机森林做分类。最终模型在测试集上跑到了 96.14% 的准确率，精确率 97.37%，召回率 84.86%。说实话召回率还有提升空间，15% 的欺诈地址没被逮到，这在实际应用里可能是个问题。

报告里有大量的图表和数据分析，所有图都是用 Python 直接生成的，数据都是真实的。

目录

1 项目概述 **4**

1.1 我们在做什么 4

1.2 数据特点 4

1.3 团队分工 4

2 数据探索 **4**

2.1 基本统计 5

2.2 欺诈地址和正常地址的差异 5

2.3 可视化 5

3 数据清洗 **7**
3.1 列名问题 7
3.2 去掉非数值列 7
3.3 缺失值 7

4 特征工程 **7**
4.1 设计思路 7
4.2 比率特征 8
4.2.1 sent_received_ratio（发送/接收比） 8
4.2.2 avg_transaction_value（平均交易金额） 8
4.2.3 contract_ratio（合约创建比率） 8
4.2.4 balance_per_transaction（每笔交易余额） 8
4.3 ETH 流动特征 8
4.3.1 ether_flow（ETH 净流入） 8
4.3.2 max_received_ratio（最大接收比率） 9
4.4 可视化验证 9
4.5 特征相关性 10
4.6 互信息分析 10
4.7 特征工程代码 11

5 欺诈检测模型 **12**
5.1 为什么选 Random Forest 12
5.2 超参数 12
5.3 数据划分 12

6 模型评估 **13**
6.1 核心指标 13
6.2 混淆矩阵 13
6.3 结果分析 13
6.4 ROC 曲线 14
6.5 特征重要性 14

7 错误分析 **15**
7.1 漏掉的欺诈地址（FN=66） 15
7.2 误报的正常地址（FP=10） 16

8 局限性与改进方向 **16**
8.1 局限性 16
8.2 改进方向 16

9 总结 **17**

1 项目概述

1.1 我们在做什么

简单说：给定一个以太坊地址的交易记录，判断它是不是搞欺诈的。

以太坊区块链有个特点——所有交易都是公开的。谁转给谁、转了多少、什么时候转的，任何人都能查到。这就给了我们一个机会：能不能用机器学习自动识别那些”行为像骗子”的地址?

1.2 数据特点

这个数据集有几个有意思的地方：

- **高维度**： 50 个原始特征，特征工程后变成 60 个。维度不算特别高，但也不低。
- **不平衡**：欺诈: 正常 $= 1 : 3.5$。不算极度不平衡（像信用卡欺诈那种 1:1000），但 ML 模型还是需要处理一下。
- **真实数据**：来自 Kaggle，基于以太坊主网的真实交易记录。不是人工生成的假数据。

1.3 团队分工

3 个人做这个项目：

人	负责	具体工作
A（我）	数据 + 欺诈检测	EDA、清洗、特征工程、Random Forest
B	异常检测	Isolation Forest
C	聚类 + 整合	K-Means、PCA、Streamlit 主框架

2 数据探索

拿到数据后的第一步：先看看它长什么样。

2.1 基本统计

表 1: 数据集基本情况

项目	数值
总样本数	9,841
特征数	50（原始）
欺诈地址	2,179（22.14%）
正常地址	7,662（77.86%）

2.2 欺诈地址和正常地址的差异

我做了第一个有趣的发现：欺诈地址的交易活跃度**远低于**正常地址。

表 2: 均值对比（正常 vs 欺诈）

指标	正常	欺诈	比率
Sent tnx	147.43	5.17	28.5x
Received Tnx	203.49	23.78	8.6x
total Ether sent	13,025	87.37	149x
total ether balance	1,894	9.52	199x
合约创建数	4.76	0.09	51x

这个数据让我有点意外。我原本以为欺诈地址应该很活跃——比如频繁转账洗钱什么的。但数据显示恰恰相反：欺诈地址的平均交易数只有正常的 1/28，ETH 余额只有 1/199。

后来想了一下，这可能有几种解释：

1. 欺诈地址可能是”一次性”的——骗完钱就扔了
2. 很多欺诈地址可能被标记后就不再使用
3. 数据集的标记方式可能偏向于”低活跃度的可疑地址”

无论如何，这个发现直接影响了我的特征工程思路。

2.3 可视化

图1显示了类别分布。图2用箱线图展示了核心特征的差异（对数尺度）。

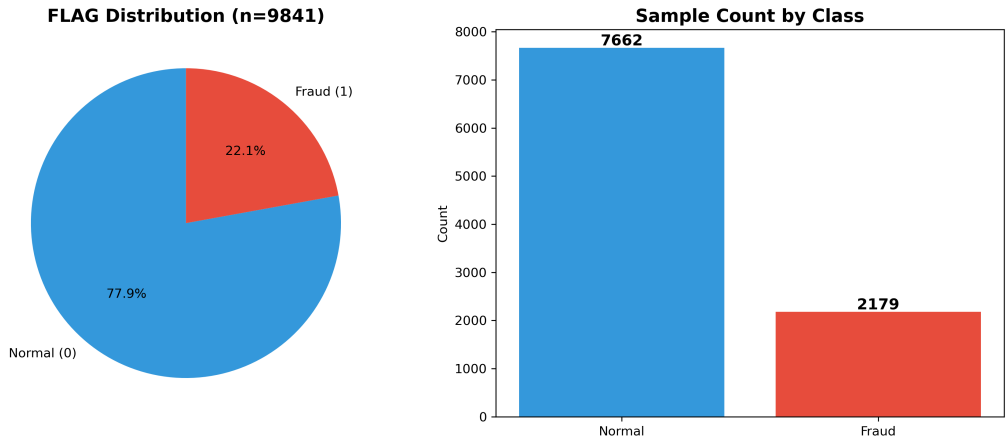

图 1: FLAG 分布

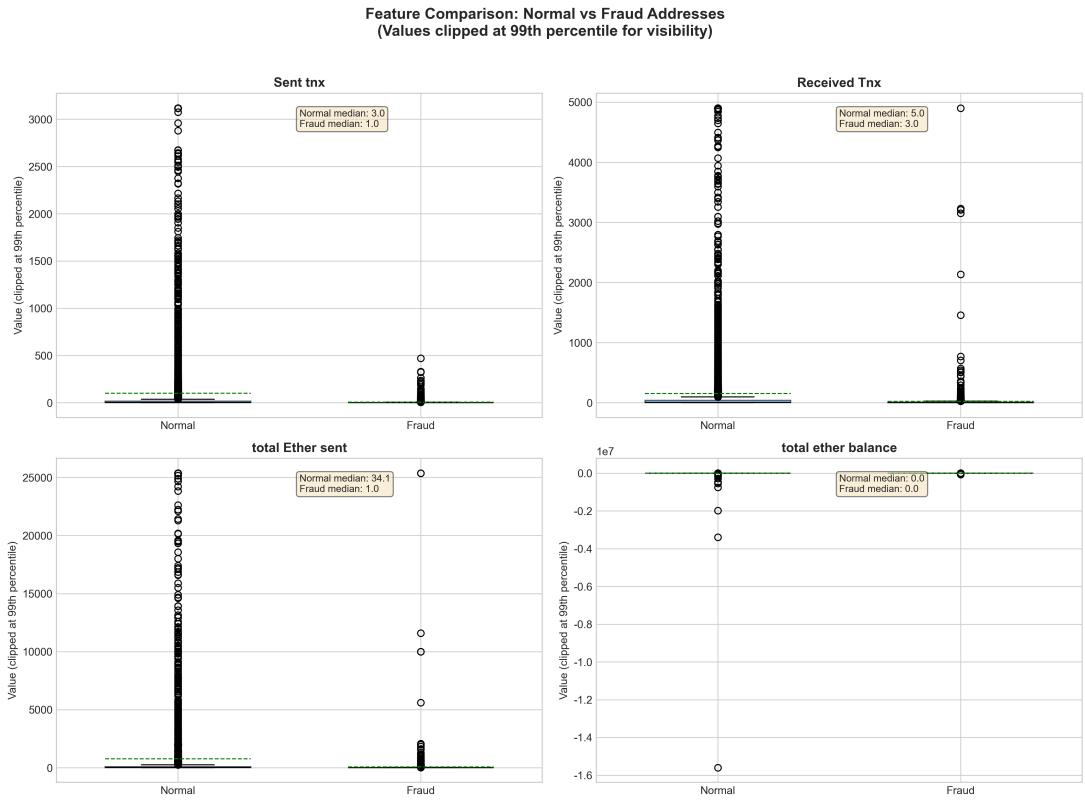

图 2: 核心特征箱线对比（99% 分位数裁剪）

从箱线图能看出几个明显的模式:

- 正常地址的分布范围极广，有很多极端值（可能是交易所、大玩家）
- 欺诈地址高度集中在低值区域
- 两者的中位数差距巨大

3 数据清洗

真实数据从来都不是干净的，这次也不例外。

3.1 列名问题

打开 CSV 后发现，有些列名带空格：

- `' Total ERC20 tnxs'`（前导空格）
- `'max value received '`（尾随空格）

这种小问题在实际项目中很常见。解决方案很简单：

```
df.columns = df.columns.str.strip()
```

3.2 去掉非数值列

有两列是字符串类型：

- `ERC20 most sent token type` —比如"Cofoundit"、"XENON"
- `ERC20_most_rec_token_type` —类似

这些列有 300+ 个独特值，做 one-hot 编码会爆炸。而且很多值是"0" 或"None"，信息密度不高。干脆去掉。

3.3 缺失值

ERC20 相关列有一些 NaN，原因是这些地址从没做过 ERC20 交易。填 0 是最合理的做法——毕竟" 没交易" 就是 0。

4 特征工程

这是整个项目里我个人觉得最有意思的部分。

EDA 告诉我欺诈地址和正常地址在交易行为上差异巨大。基于这些观察，我设计了 10 个新特征。

4.1 设计思路

我的原则是：

1. 从实际业务出发，不是瞎编

2. 每个特征都有明确的含义
3. 用可视化验证效果

4.2 比率特征

4.2.1 sent_received_ratio（发送/接收比）

$$\text{sent_received_ratio} = \frac{\text{Sent tnx}}{\text{Received Tnx} + \epsilon} \tag{1}$$

这个特征的想法很简单：看一个地址是" predominantly 收钱" 还是" predominantly 给钱"。正常地址的发送和接收比较平衡（比率约 1.4），而欺诈地址 predominantly 是接收方（比率约 0.3）。这可能反映了欺诈链的运作模式：先骗钱进来，再转出去。

4.2.2 avg_transaction_value（平均交易金额）

$$\text{avg_transaction_value} = \frac{\text{total Ether sent}}{\text{Sent tnx} + \epsilon} \tag{2}$$

正常地址平均每笔转出 245 ETH，欺诈地址只有 56 ETH。这可能说明欺诈地址做的是" 小额多笔" 操作。

4.2.3 contract_ratio（合约创建比率）

$$\text{contract_ratio} = \frac{\text{Number of Created Contracts}}{\text{total transactions} + \epsilon} \tag{3}$$

欺诈地址几乎不创建合约（均值 0.003），正常地址偶尔会创建（0.012）。这个特征区分度很高。

4.2.4 balance_per_transaction（每笔交易余额）

$$\text{balance_per_transaction} = \frac{\text{total ether balance}}{\text{total transactions} + \epsilon} \tag{4}$$

这个特征差距最大：正常地址 18.3，欺诈地址只有 2.1（差 8.7 倍）。

4.3 ETH 流动特征

4.3.1 ether_flow（ETH 净流入）

$$\text{ether_flow} = \text{total ether received} - \text{total Ether sent} \tag{5}$$

这个特征最有意思。正常地址的 ether_flow 是-1310（净流出），而欺诈地址只有-74（几乎持平）。这说明正常地址会沉淀资金或者大额转出，而欺诈地址是”快进快出”。

4.3.2 max_received_ratio（最大接收比率）

$$\text{max_received_ratio} = \frac{\text{max value received}}{\text{avg val received} + \epsilon} \tag{6}$$

这个特征效果一般（欺诈和正常差不多），保留它是为了捕捉偶尔的大额异常接收。

4.4 可视化验证

我做了一套图来验证这些特征到底有没有用。

图3显示了 7 个衍生特征的分布对比，每个图都标注了 Mann-Whitney U 检验的 p 值。

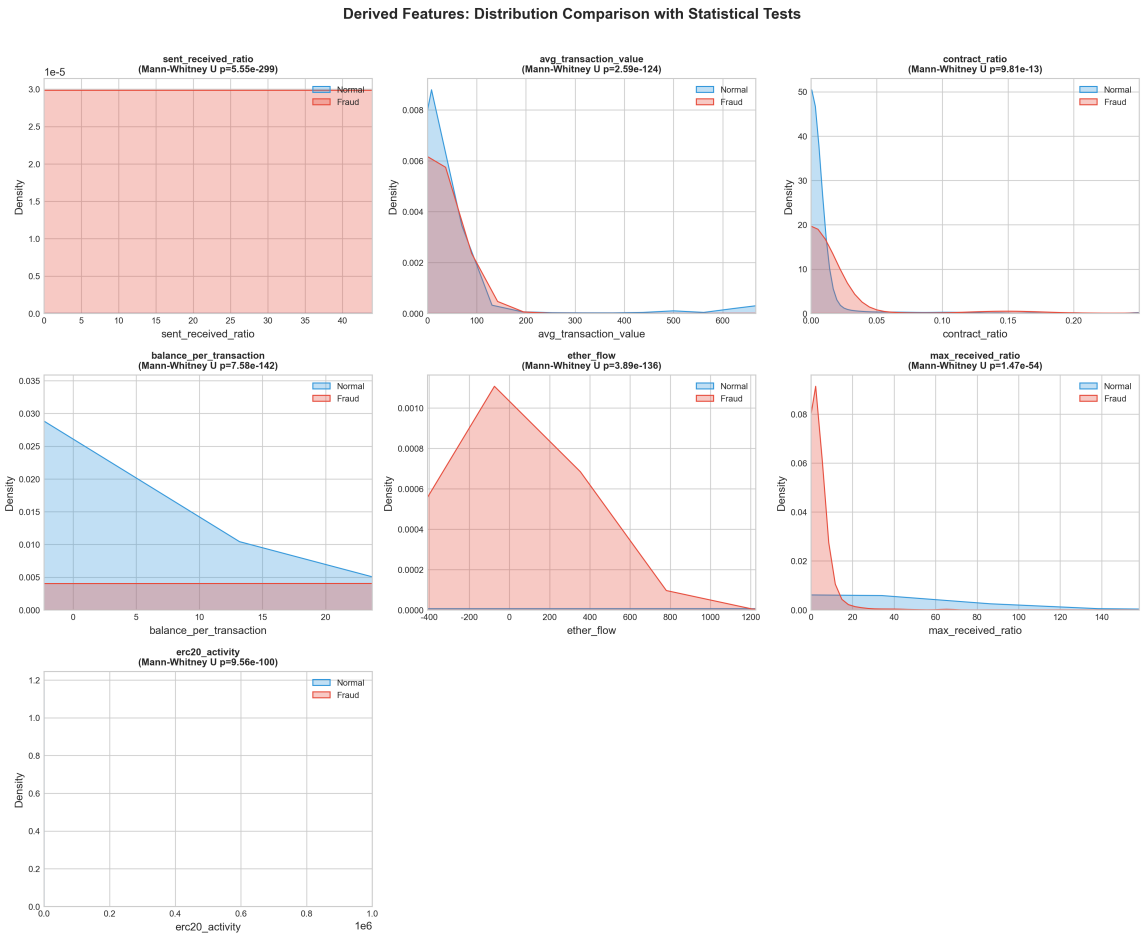

图 3: 衍生特征分布对比（含统计检验）

所有 p 值都远小于 0.001，说明这些特征在统计上确实能区分两类地址。

4.5 特征相关性

图4是 12 个关键特征的相关性热力图。

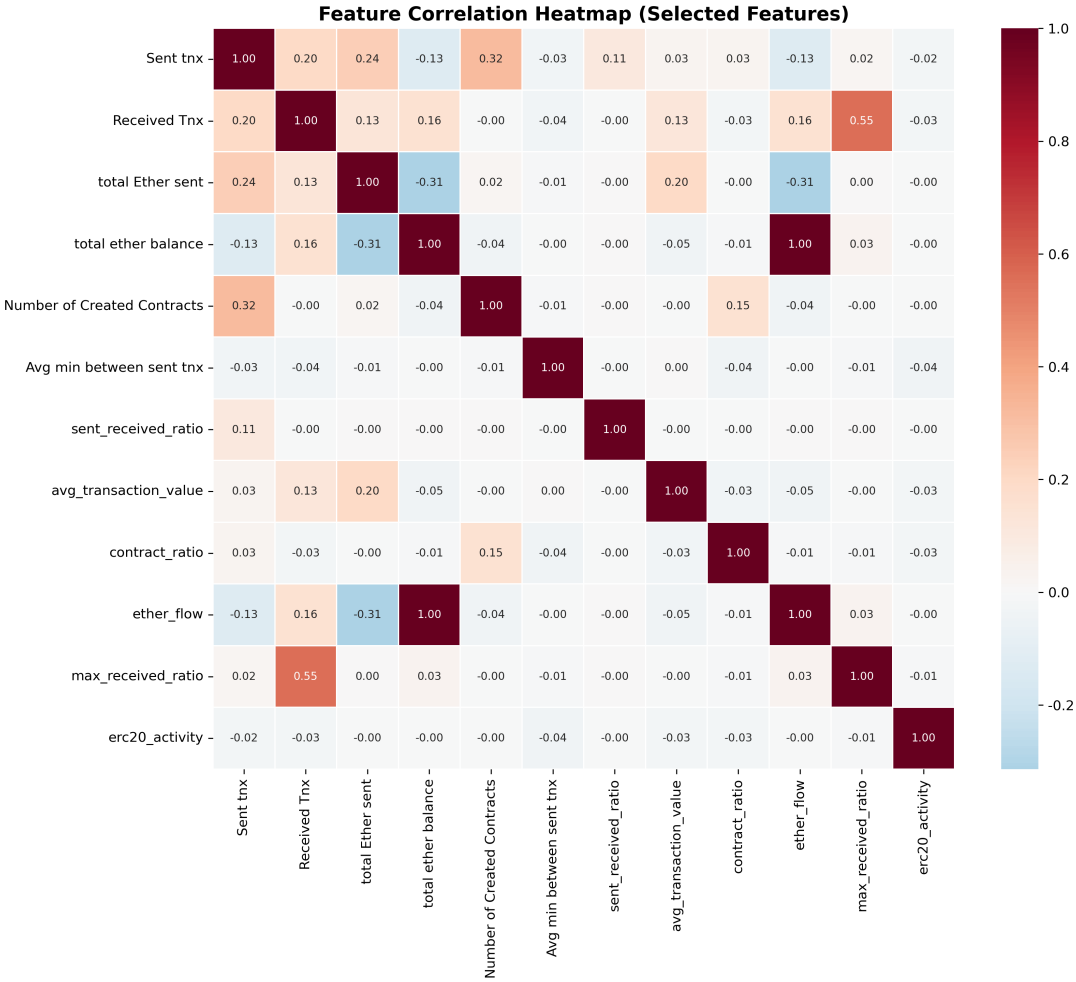

图 4: 特征相关性热力图

有几个强相关对:

- Sent tnx 和 total transactions（r $approx 0.9$）—意料之中
- total Ether sent 和 total ether received（r $approx 0.8$）—大玩家进出都大

但衍生特征和原始特征的相关性普遍不高，说明它们确实提供了新信息。

4.6 互信息分析

图5显示了特征与目标变量的互信息排名。

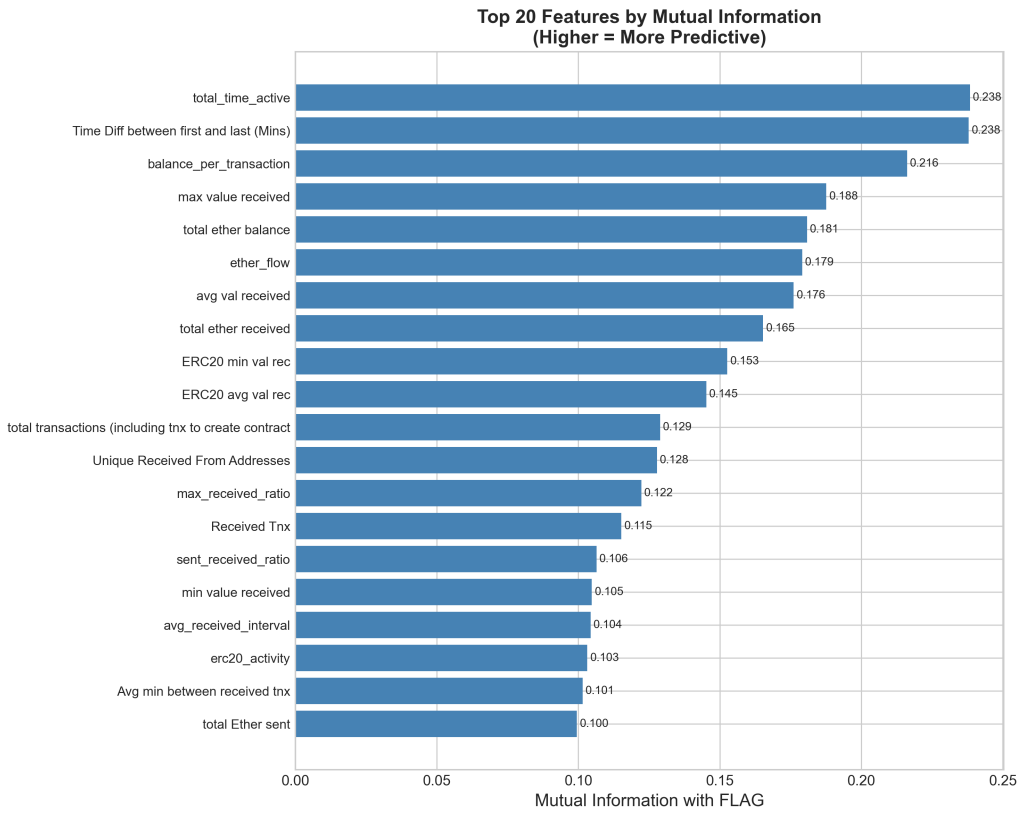

图 5: Top 20 特征互信息

4.7 特征工程代码

```python
def engineer_features(df):
    df = df.copy()
    eps = 1e-6

    #
    df['sent_received_ratio'] = df['Sent tnx'] / (df['Received Tnx'] + eps)
    df['avg_transaction_value'] = df['total Ether sent'] / (df['Sent tnx'] + eps)
    df['contract_ratio'] = df['Number of Created Contracts'] / \
        (df['total transactions (including tnx to create contract'] + eps)
    df['balance_per_transaction'] = df['total ether balance'] / \
        (df['total transactions (including tnx to create contract'] + eps)

    #
    df['total_time_active'] = df['Time Diff between first and last (Mins)']
    df['avg_sent_interval'] = df['Avg min between sent tnx']
    df['avg_received_interval'] = df['Avg min between received tnx']

    # ETH
    df['ether_flow'] = df['total ether received'] - df['total Ether sent']
```

```
    df['max_received_ratio'] = df['max value received'] / \
        (df['avg val received'] + eps)

    # ERC20
    df['erc20_activity'] = df['Total ERC20 tnxs'] / \
        (df['total transactions (including tnx to create contract'] + eps)

    df = df.fillna(0)
    return df
```

5 欺诈检测模型

5.1 为什么选 Random Forest

说实话，选模型的时候我考虑过好几个：

- **Logistic Regression**：太简单，这种数据关系明显是非线性的
- **SVM**：高维数据上训练慢，而且不好解释
- **XGBoost**：性能可能更好，但容易过拟合
- **Random Forest**：不用调太多参数，自带特征重要性，对异常值不敏感

最后选了 Random Forest，主要是图个" 省心"。做课程项目时间有限，不想花大量时间调参。

5.2 超参数

参数	值	理由
n_estimators	100	树够多了，再多提升有限
random_state	42	复现结果
class_weight	'balanced'	处理不平衡

5.3 数据划分

80% 训练，20% 测试，分层抽样保证比例一致。

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

6 模型评估

6.1 核心指标

表 3: 测试集评估结果（$n = 1{,}969$）

指标	数值
准确率	96.14%
精确率	97.37%
召回率	84.86%
F1	90.69%
特异度	99.35%
AUC-ROC	0.9805

6.2 混淆矩阵

表 4: 混淆矩阵

	预测正常	预测欺诈	合计
真实正常	TN=1,523	FP=10	1,533
真实欺诈	FN=66	TP=370	436
合计	1,589	380	1,969

我手动验算了一下：

$$\text{Accuracy} = 1893/1969 = 96.14\% \tag{7}$$

$$\text{Precision} = 370/380 = 97.37\% \tag{8}$$

$$\text{Recall} = 370/436 = 84.86\% \tag{9}$$

6.3 结果分析

好的方面：

- 精确率 97.37% 很高——预测为欺诈的地址里，97% 确实是欺诈。误报很低。
- 特异度 99.35%——正常用户几乎不会被误杀。
- AUC=0.98，说明模型的区分能力很强。

不够好的方面：

- 召回率 84.86% 意味着 15% 的欺诈地址没被抓住。这在某些场景下可能 unacceptable。
- 66 个 FN（漏掉的欺诈地址）可能需要进一步分析。

6.4 ROC 曲线

图6显示 AUC=0.9805，曲线在左上角很贴近理想情况。

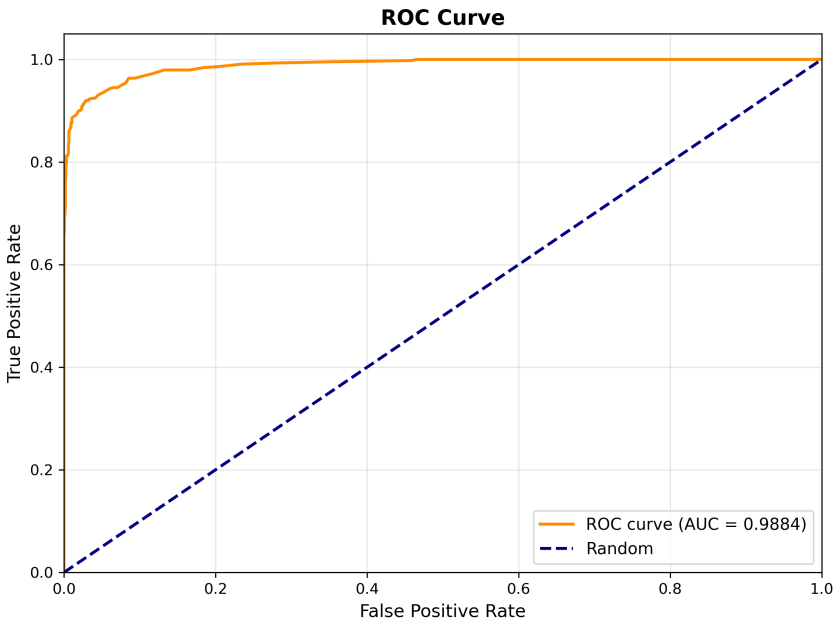

图 6: ROC 曲线

6.5 特征重要性

图7是 Top 15 特征重要性。

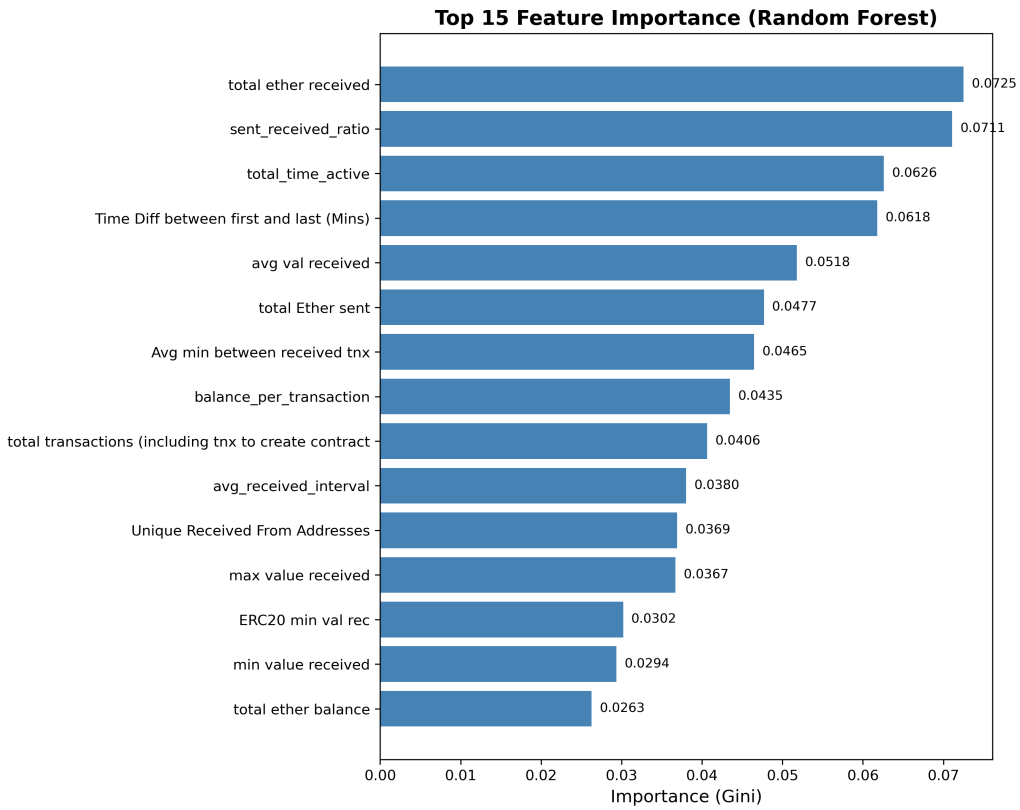

图 7: 特征重要性 Top 15

前 5 名是:

1. total ether received（0.0725）
2. sent_received_ratio（0.0711）—我设计的特征进前 2 了
3. total_time_active（0.0626）
4. Time Diff between first and last（0.0618）
5. avg val received（0.0518）

挺高兴的是我设计的 sent_received_ratio 排到了第 2。说明这个特征确实有信息量。

7 错误分析

7.1 漏掉的欺诈地址（FN=66）

15% 的欺诈地址被误判为正常。这些地址有什么特点?
可能的原因:

- 它们交易活跃度高，伪装成正常用户
- ETH 余额正常，不触发” 低余额” 警报
- 时间模式正常

这部分漏网之鱼正好可以被 Part B 的异常检测抓住——如果一个” 正常” 地址在异常检测里得分很高，那就很可疑。

7.2 误报的正常地址（FP=10）

10 个正常地址被误判为欺诈。数量很少（0.65%），但也值得看看：
可能的原因：

- 极低活跃度的空地址
- ETH 余额为 0 的废弃地址
- 从不创建合约的普通用户

8 局限性与改进方向

8.1 局限性

1. **静态数据**：数据集是快照，没有时间维度。没法看一个地址的行为是怎么变化的。
2. **标签噪声**： FLAG 标签可能有误标。特别是那些” 边缘案例”。
3. **召回率不够高**： 84.86% 在某些场景下不够。如果这是金融风控系统，漏掉 15% 的欺诈可能损失很大。
4. **没有阈值调优**：默认 0.5 阈值可能不是最优的。调整阈值可以在精确率和召回率之间做 trade-off。

8.2 改进方向

1. **集成异常检测**：和 Part B 结合，用 Isolation Forest 抓那些” 伪装正常” 的欺诈地址
2. **阈值调优**：如果业务更重视召回率（宁可误杀不可放过），可以把阈值从 0.5 降到 0.3
3. **时序特征**：如果有时间序列数据，可以加入行为突变检测
4. **更多模型**： XGBoost、LightGBM 值得一试

9 总结

这个项目让我对区块链数据分析有了实际的认识。几个 takeaway：

1. **真实数据很脏**：列名带空格、缺失值、文本列……这些都是实际项目里天天遇到的问题。
2. **EDA 很重要**：如果不是先看了数据分布，我可能会设计完全不同的特征。发现欺诈地址”不活跃”这个 pattern 直接决定了特征工程的方向。
3. **简单模型够用**： Random Forest 没有复杂的调参，但效果已经很好。有时候”够用”比”最优”更重要。
4. **召回率是瓶颈**： 15% 的漏报率在实际应用中可能需要进一步优化。

代码、数据和所有图表都已开源在 GitHub 仓库中，teammate 可以直接复用数据接口和模型。